Лабораторна робота №11
Робота з СКБД MySQL та оболонкою phpMyAdmin

Крайняк Михайло Юрійович, 451 група

1. Тема, мета, середовище

Тема: класичні засоби роботи з MySQL: SQL, phpMyAdmin і процедурний інтерфейс MySQLi у PHP.

Мета: навчитися створювати таблицю, виконувати SELECT, INSERT, UPDATE, DELETE, отримувати результати запитів і переносити каталог із XML/JSON до реляційної бази; закріпити перевірку введення, параметризовані запити та безпечне відображення даних.

Середовище: XAMPP, **PHP 8.2.12** (розширення `mysqli`, `mbstring`); **MariaDB 10.11.19**; **phpMyAdmin 5.2.3**. Компонента MySQL у встановленому XAMPP немає, тому MariaDB і phpMyAdmin узято з офіційних архівів (mariadb.org, phpmyadmin.net, контрольні суми SHA-256 перевірено) і запущено лише на 127.0.0.1: MariaDB — порт 3306, phpMyAdmin — вбудований сервер PHP на порту 8080, застосунок — `php -S 127.0.0.1:8000 -t lab-11` (адреси `/src/…`, як `http://localhost/lab-11/src/` в XAMPP).

Варіант 10 (той самий, що в ЛР 8): розклад маршрутів автовокзалу.

2. Готовий і власний код

Файл	**Походження**	**Призначення**
`src/schema.sql`, `src/common.php`, `src/index.php`, `src/crud-demo.php`	Готові, **без змін**	Таблиця products, підключення `connectDatabase()`, `validateProduct()`, `findProducts()`, сторінка читання, CLI-цикл CRUD
`src/app.php`	Власне	Сесія з cookie-шляхом проєкту, CSRF, `idFrom()`, 303, flash, нейтральна обробка збоїв БД, шаблон сторінок, `findProductById()`
`src/products.php`, `product-form.php`, `product-delete.php`	Власне	Крок 7: CRUD товарів через форми
`src/fetch-demo.php`	Власне	Крок 4: порівняння fetch-функцій
`sql/routes-schema.sql`, `src/routes-common.php`, `routes.php`, `route-form.php`, `route-delete.php`	Власне	Крок 8: таблиця `bus_routes`, список, точний пошук, додавання, редагування, видалення
`src/routes-import.php`, `data/routes-lab08.json|xml`	Власне	Крок 8.3: імпорт каталогу ЛР 8 у транзакції

Файл	Походження	Призначення
`sql/app-user.sql`, `env.example`	Власне	Крок 1: база й користувач застосунку (пароль — заглушка), перелік змінних середовища
`export/lab11.sql`, `sql/routes-checks.sql`, `tools/restore-check.txt`	Власне	Крок 9: експорт phpMyAdmin, доповнення для CHECK-обмежень, порівняння відновлення
`tools/checks.sh` та `*-output.txt`	Власне	Автоматичні перевірки й фактичні результати

3. Хід роботи

Крок 1. Середовище, база й користувач застосунку

У phpMyAdmin (вкладка SQL) створено базу `lab11` з `utf8mb4_unicode_ci` і користувача `lab11_app` (для `localhost` та `127.0.0.1`) лише з правами SELECT, INSERT, UPDATE, DELETE на `lab11`. Структуру створює адміністратор. Пароль користувача згенеровано випадково й зберігається лише в локальних змінних середовища — у PHP-файлах, звіті й репозиторії його немає (шаблон — `sql/app-user.sql`).

```
GRANT USAGE ON *.* TO `lab11_app`@`127.0.0.1` IDENTIFIED BY PASSWORD '<прибрано>'
GRANT SELECT, INSERT, UPDATE, DELETE ON `lab11`.* TO `lab11_app`@`127.0.0.1`
```

Крок 2. Таблиця products і запити

Готовий `schema.sql` виконано у вкладці SQL phpMyAdmin для бази `lab11`.

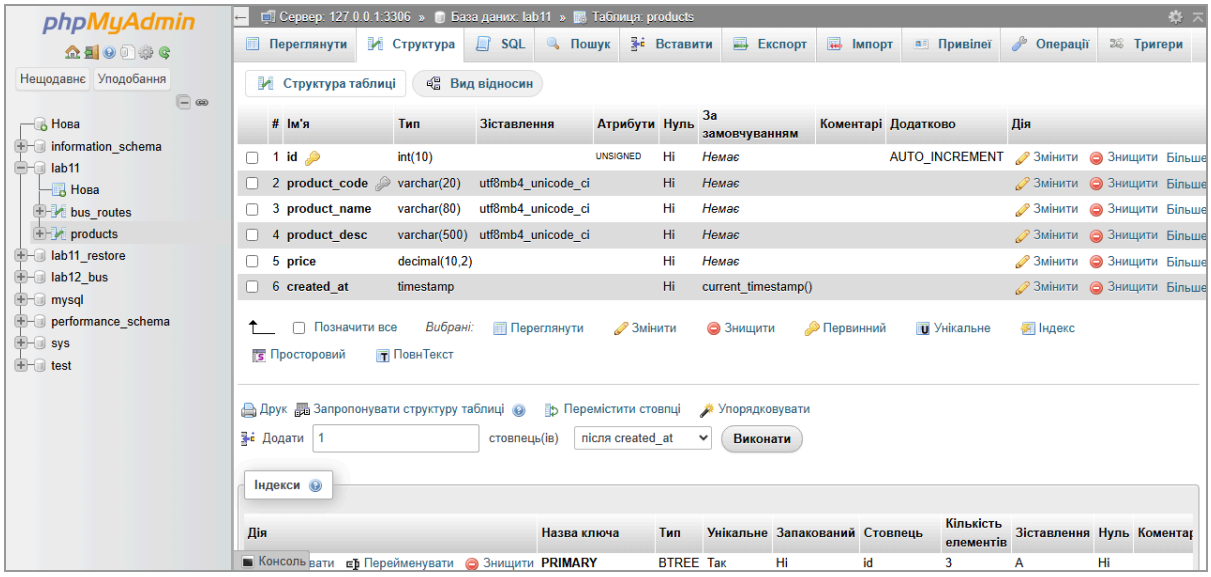


Рис. 1. phpMyAdmin → `lab11.products` → «Структура»: `id` — INT UNSIGNED AUTO_INCREMENT і PRIMARY, `product_code` — UNIQUE `uq_products_code`, `price` — DECIMAL(10,2), `created_at` — CURRENT_TIMESTAMP, усі поля NOT NULL, кодування utf8mb4_unicode_ci.

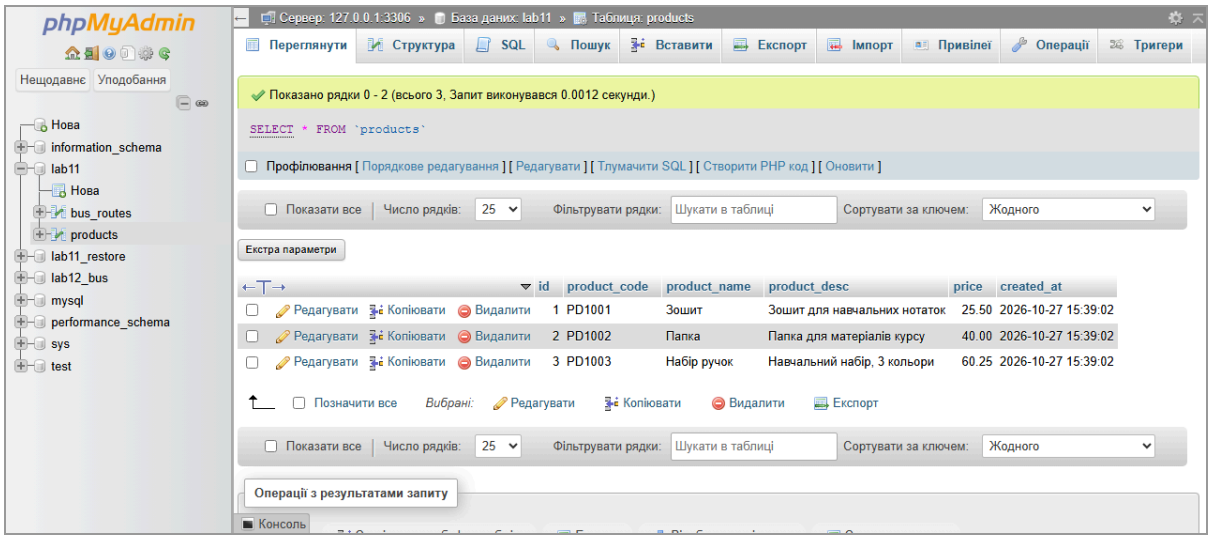


Рис. 2. «Переглянути»: три товари — Зошит 25.50, Папка 40.00, Набір ручок 60.25.

```
-- Крок 2: запити у базі lab11

> SELECT id, product_code, product_name, price FROM products ORDER BY id;
+----+--------------+-----------------------+-------+
| id | product_code | product_name          | price |
+----+--------------+-----------------------+-------+
|  1 | PD1001       | Зошит                 | 25.50 |
|  2 | PD1002       | Папка                 | 40.00 |
|  3 | PD1003       | Набір ручок           | 60.25 |
+----+--------------+-----------------------+-------+

> SELECT COUNT(*) AS total FROM products;
+-------+
| total |
+-------+
|     3 |
+-------+

> SELECT product_name, price FROM products WHERE price >= 40.00 ORDER BY price, id;
+-----------------------+-------+
| product_name          | price |
+-----------------------+-------+
| Папка                 | 40.00 |
| Набір ручок           | 60.25 |
+-----------------------+-------+

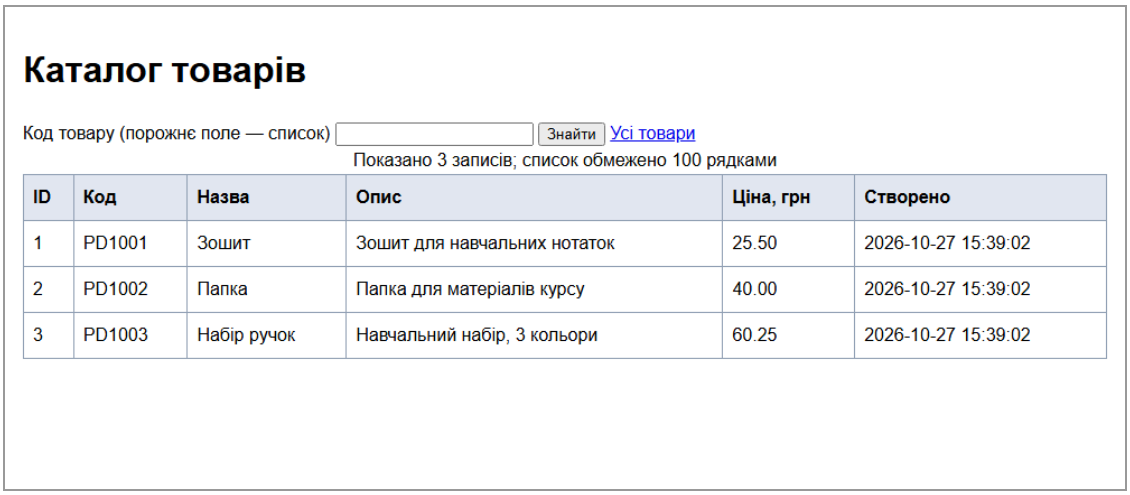
-- Крок 1.4: права користувача застосунку
GRANT USAGE ON *.* TO `lab11_app`@`127.0.0.1` IDENTIFIED BY PASSWORD '<прибрано>'
GRANT SELECT, INSERT, UPDATE, DELETE ON `lab11`.* TO `lab11_app`@`127.0.0.1`
GRANT SELECT ON `lab11_restore`.* TO `lab11_app`@`127.0.0.1`
```

Висновок. `COUNT(*)` рахує рядки (3), `WHERE price >= 40.00` відбирає 2 товари, `ORDER BY price, id` задає порядок (за ціною, за однакової — за id). Без ORDER BY порядок рядків не гарантований. Значення `id` генерує AUTO_INCREMENT; пропуски в них нормальні.

Крок 3. Підключення PHP

Готова `connectDatabase()` читає `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` із середовища процесу PHP, вмикає `MYSQLI_REPORT_STRICT` (помилки — винятки), тайм-аут 5 с і `utf8mb4`. Змінні задано для вебсервера і CLI окремо; перелік — у `env.example` без пароля.

Крок 4. Читання каталогу й fetch-функції



Каталог товарів

Код товару (порожнє поле — список) Знайти Усі товари

Показано 3 записів; список обмежено 100 рядками

ID	Код	Назва	Опис	Ціна, грн	Створено
1	PD1001	Зошит	Зошит для навчальних нотаток	25.50	2026-10-27 15:39:02
2	PD1002	Папка	Папка для матеріалів курсу	40.00	2026-10-27 15:39:02
3	PD1003	Набір ручок	Навчальний набір, 3 кольори	60.25	2026-10-27 15:39:02

Рис. 3. Готова сторінка `index.php`: три товари з таблиці.

Вивід `php fetch-demo.php` (власний фрагмент; перед кожним способом курсор повертається `mysqli_data_seek($result, 0)`):

```
$ php fetch-demo.php
mysqli_fetch_assoc — доступ за назвою стовпця:
  1: Зошит — 25.50
  2: Папка — 40.00
  3: Набір ручок — 60.25
  ще один fetch після циклу: NULL (курсор у кінці)

mysqli_fetch_row — доступ за індексом (назва — третій стовпець, [2]):
  1: Зошит — 25.50
  2: Папка — 40.00
  3: Набір ручок — 60.25

mysqli_fetch_object — властивості об’єкта stdClass:
  1: Зошит — 25.50
  2: Папка — 40.00
  3: Набір ручок — 60.25

mysqli_fetch_array(MYSQLI_BOTH) — і назва, і індекс:
  ['product_name'] = Зошит, [2] = Зошит, ключів у рядку: 8 (4 стовпці × 2)
  типи значень: id — string, price — string (DECIMAL повертається рядком)

Порожній результат: mysqli_num_rows = 0, fetch = NULL — це не помилка SQL.
```

Крок 5. Параметризований пошук

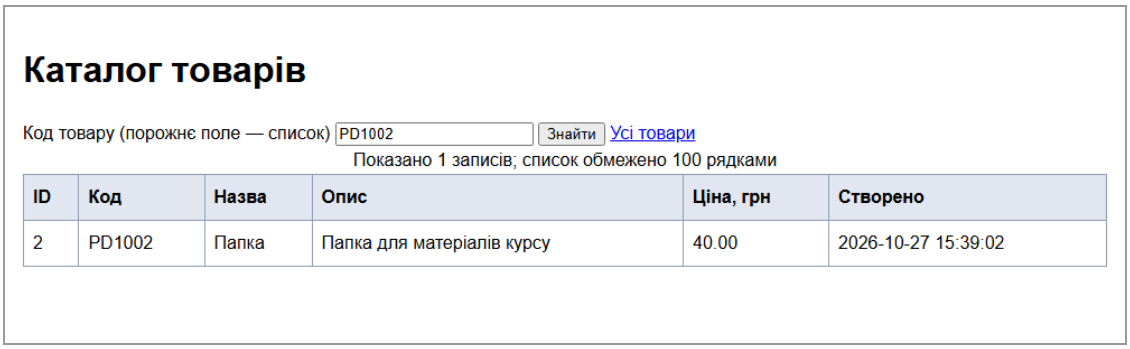


Рис. 4. Пошук `PD1002` (підготовлений запит з `?` і типом `s`): лише «Папка», 40.00. Порожнє поле показує весь список, `NO-SUCH-CODE` — «Товарів не знайдено.», `code[]=PD1002` — 400 без попереджень (див. таблицю результатів).

Крок 6. Цикл CRUD у CLI

```
$ php crud-demo.php
INSERT → SELECT → UPDATE → SELECT → DELETE перевірено; транзакцію відкочено.
```

Після запуску в таблиці, як і раніше, три початкові товари. Наступний id після відкату може мати пропуск — AUTO_INCREMENT не повертається.

Крок 7. Власні форми для товарів

GET лише показує список або форму; INSERT, UPDATE і DELETE виконує тільки POST після перевірки CSRF-токена й готової `validateProduct()`. Під час редагування рядок спершу блокується `SELECT … FOR UPDATE` у транзакції: невідомий id дає 404 і не створює запис, а `affected_rows = 0` для наявного рядка означає «збережено без змін». Дублікат коду (помилка 1062) показується як помилка форми зі збереженими значеннями; інші збої — нейтральним повідомленням, код пишеться в журнал.

```
            mysqli_begin_transaction($db);
            $inTransaction = true;
            // Блокуємо рядок і переконуємося, що він існує: невідомий id не створює товар.
            $stmt = mysqli_prepare($db, 'SELECT id FROM products WHERE id = ? FOR UPDATE');
            mysqli_stmt_bind_param($stmt, 'i', $id);
            mysqli_stmt_execute($stmt);
            mysqli_stmt_store_result($stmt);
            $exists = mysqli_stmt_num_rows($stmt) === 1;
            mysqli_stmt_close($stmt);
            if (!$exists) {
                throw new DomainException('Товар не знайдено; новий запис не створено.', 404);
            }
            $stmt = mysqli_prepare($db,
                'UPDATE products SET product_code = ?, product_name = ?, product_desc = ?, price = ? WHERE
id = ?');
            mysqli_stmt_bind_param($stmt, 'ssssi', $product['product_code'], $product['product_name'],
                $product['product_desc'], $product['price'], $id);
            mysqli_stmt_execute($stmt);
            $changed = mysqli_stmt_affected_rows($stmt);
            mysqli_stmt_close($stmt);
            mysqli_commit($db);
```

```php
/** Перевірка до будь-якого INSERT, UPDATE чи DELETE. */
function requireCsrf(): void
{
    $token = $_POST['csrf'] ?? null;
    if (!is_string($token) || !hash_equals($_SESSION['csrf'], $token)) {
        throw new DomainException('Форма застаріла (CSRF-токен не збігається). Оновіть сторінку.', 403);
    }
}

/** Збій БД: код і контекст — у журнал (без пароля та введених даних), користувачу — нейтральний текст. */
function dbFailure(Throwable $exception, string $context): string
{
    error_log('lab11 ' . $context . ': code=' . $exception->getCode());
    http_response_code(500);
    return 'База даних недоступна або сталася помилка. Зміни не виконано.';
}
```



Рис. 5. `products.php`: список із діями «Редагувати» та «Видалити».

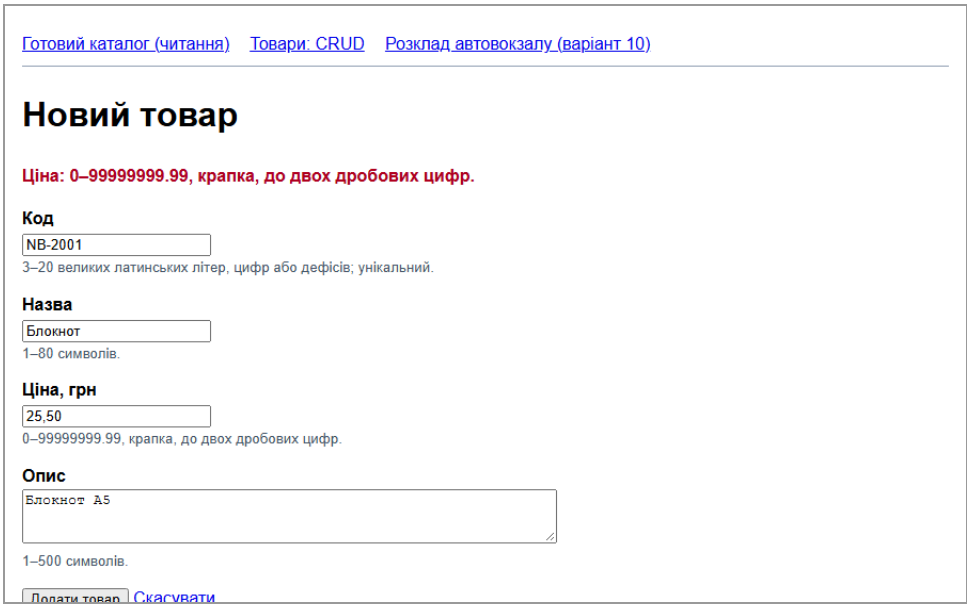


Рис. 6. Ціна `25,50` (кома) — відмова 422 до звернення до БД; введені значення збережено.

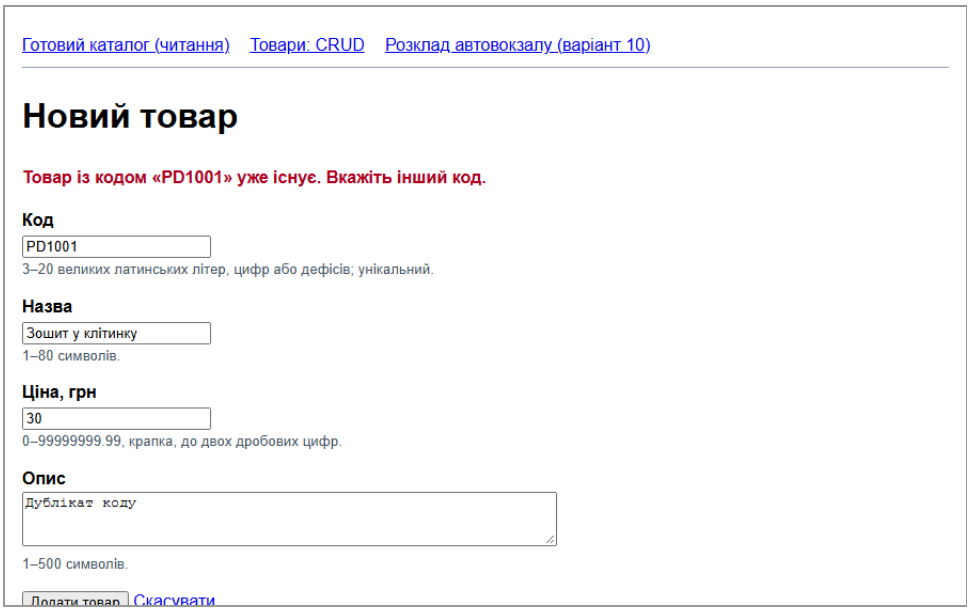


Рис. 7. Повторний код `PD1001` — конфлікт 1062 оброблено як помилку форми (409), другий товар не створено.



Рис. 8. Збереження товару з тими самими даними: перенаправлення 303 і повідомлення «збережено без змін (affected_rows = 0)».

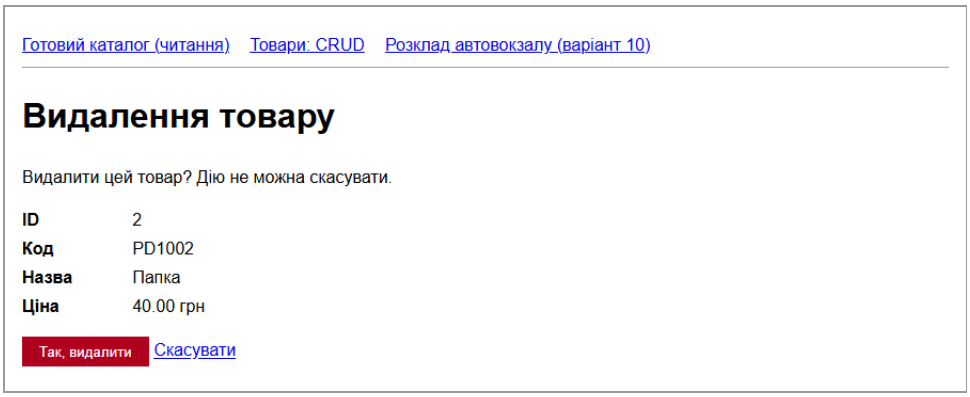


Рис. 9. Видалення з явним підтвердженням: GET лише показує запис, DELETE виконує кнопка (POST із CSRF).

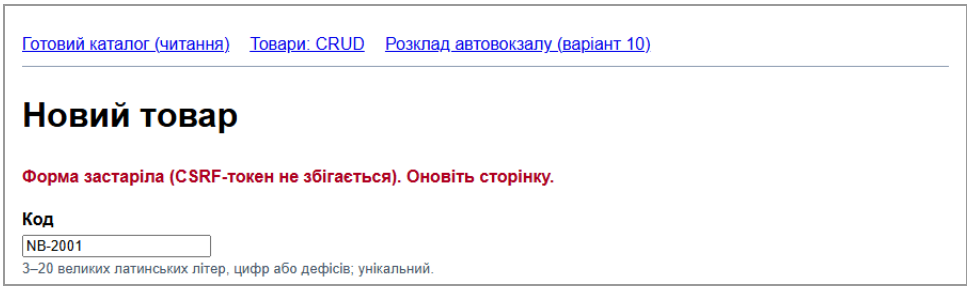


Рис. 10. POST без CSRF-токена — 403, запит до БД не виконується.

Крок 8. Індивідуальне завдання: таблиця bus_routes (варіант 10)

Поле	Тип	NOT NULL	Межі / правила застосунку
`id`	INT UNSIGNED AUTO_INCREMENT	так	PRIMARY KEY
`route_number`	VARCHAR(10)	так	1–10 літер, цифр, дефісів (`105`, `212-A`)
`destination`	VARCHAR(100)	так	2–100 символів (`mb_strlen`), без керівних символів; індекс для пошуку
`departure_time`	TIME	так	ГГ:ХХ, 00:00–23:59
`platform`	TINYINT UNSIGNED	так	1–20 (і CHECK у БД); bind_param — `i`
`price`	DECIMAL(7,2)	так	1.00–5000.00 грн (і CHECK у БД); передається рядком, тип `s`
`created_at`	TIMESTAMP	так	DEFAULT CURRENT_TIMESTAMP (встановлює сервер)

Змістова підстава для унікального індексу: той самий рейс не може відправлятися двічі в один час — `UNIQUE (route_number, departure_time)`. Структуру з п’ятьма вигаданими рейсами створює `sql/routes-schema.sql` (виконано у вкладці SQL phpMyAdmin).

```
CREATE TABLE bus_routes (
    id             INT UNSIGNED     NOT NULL AUTO_INCREMENT,
    route_number   VARCHAR(10)      NOT NULL COMMENT 'Номер рейсу: 1–10 літер, цифр, дефісів',
    destination    VARCHAR(100)     NOT NULL COMMENT 'Пункт призначення: 2–100 символів',
    departure_time TIME             NOT NULL COMMENT 'Час відправлення 00:00–23:59',
    platform       TINYINT UNSIGNED NOT NULL COMMENT 'Платформа 1–20',
    price          DECIMAL(7,2)     NOT NULL COMMENT 'Ціна квитка, грн: 1.00–5000.00',
    created_at     TIMESTAMP        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    -- Той самий рейс не може відправлятися двічі в один час.
    UNIQUE KEY uq_bus_routes_number_time (route_number, departure_time),
    KEY idx_bus_routes_destination (destination),
    CONSTRAINT chk_bus_routes_platform CHECK (platform BETWEEN 1 AND 20),
    CONSTRAINT chk_bus_routes_price CHECK (price BETWEEN 1.00 AND 5000.00)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Імпорт із ЛР 8. `routes-import.php` спершу перевіряє файл за правилами ЛР 8 (структура, типи, межі, унікальність id; для XML — DOM без DTD), потім виконує один підготовлений INSERT у циклі в межах транзакції. Політика дублікатів: якщо хоч один рейс (номер + час) уже є, відкочується вся група, тому повторний імпорт нічого не додає.

```
$ php routes-import.php json
Імпортовано з routes-lab08.json: 3 рейс(и); транзакцію підтверджено.
(код завершення 0)

$ php routes-import.php xml   # ті самі рейси повторно
Рейс із таким номером і часом уже є в таблиці; транзакцію відкочено, нічого не імпортовано.
(код завершення 1)
```

```
    $stmt = mysqli_prepare($db, 'INSERT INTO bus_routes (route_number, destination, departure_time,
platform, price)
                                 VALUES (?, ?, ?, ?, ?)');
    $number = $destination = $time = $price = '';
    $platform = 0;
    mysqli_stmt_bind_param($stmt, 'sssis', $number, $destination, $time, $platform, $price);
    foreach ($routes as $route) {
        $number = $route['number'];
        $destination = $route['destination'];
        $time = $route['departure'] . ':00';
        $platform = $route['platform'];
        $price = $route['price'] . '.00';
        mysqli_stmt_execute($stmt);
    }
    mysqli_stmt_close($stmt);
    mysqli_commit($db);
```

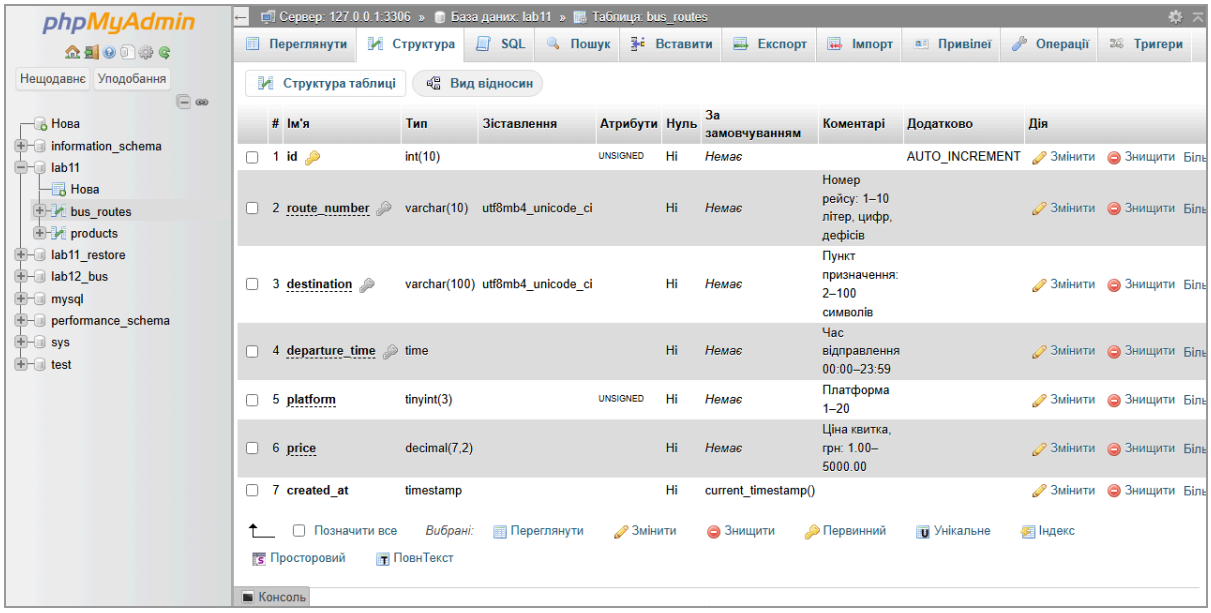


Рис. 11. phpMyAdmin → `bus_routes` → «Структура»: типи, NOT NULL, коментарі з межами.

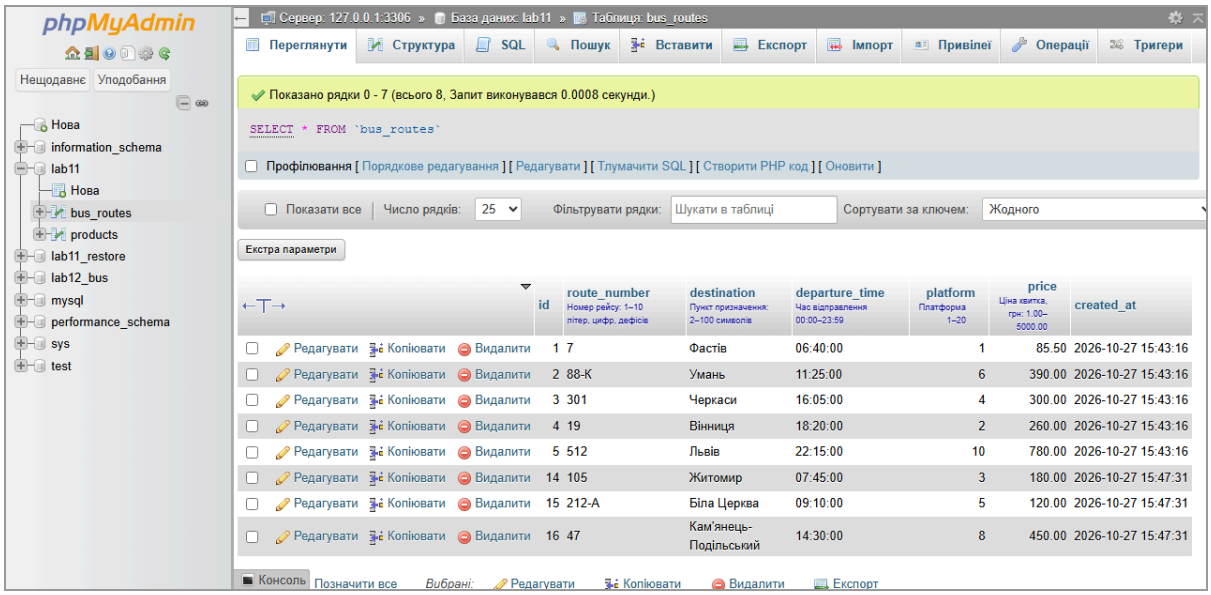


Рис. 12. «Переглянути»: 5 початкових рейсів і 3 рейси, імпортовані з ЛР 8 (Житомир, Біла Церква, Кам’янець-Подільський).

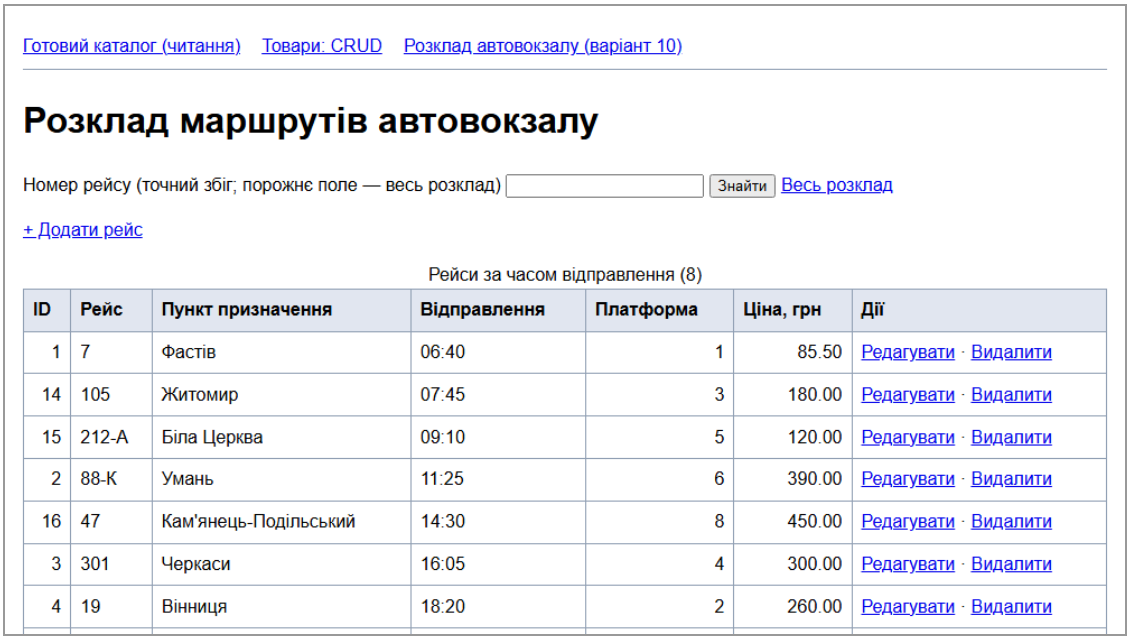


Рис. 13. `routes.php`: розклад за часом відправлення з діями.

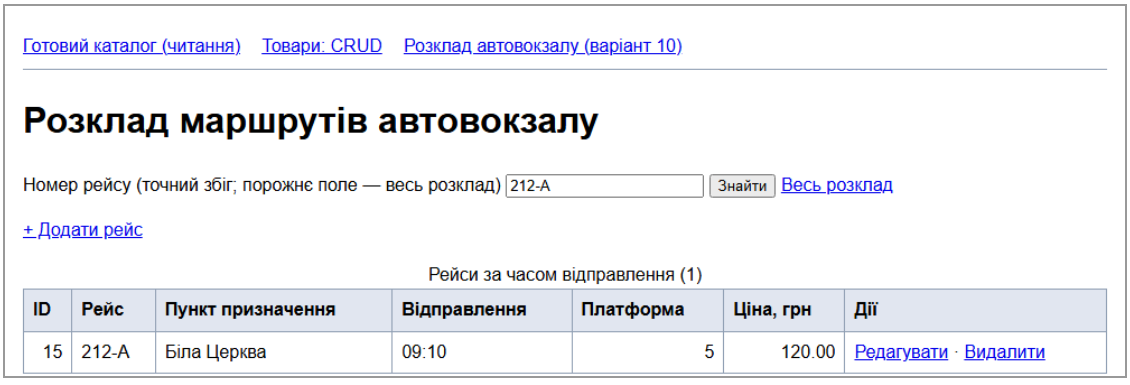


Рис. 14. Точний пошук за номером рейсу `212-A` (підготовлений запит); запит `212` дає «Рейс «212» не знайдено.».

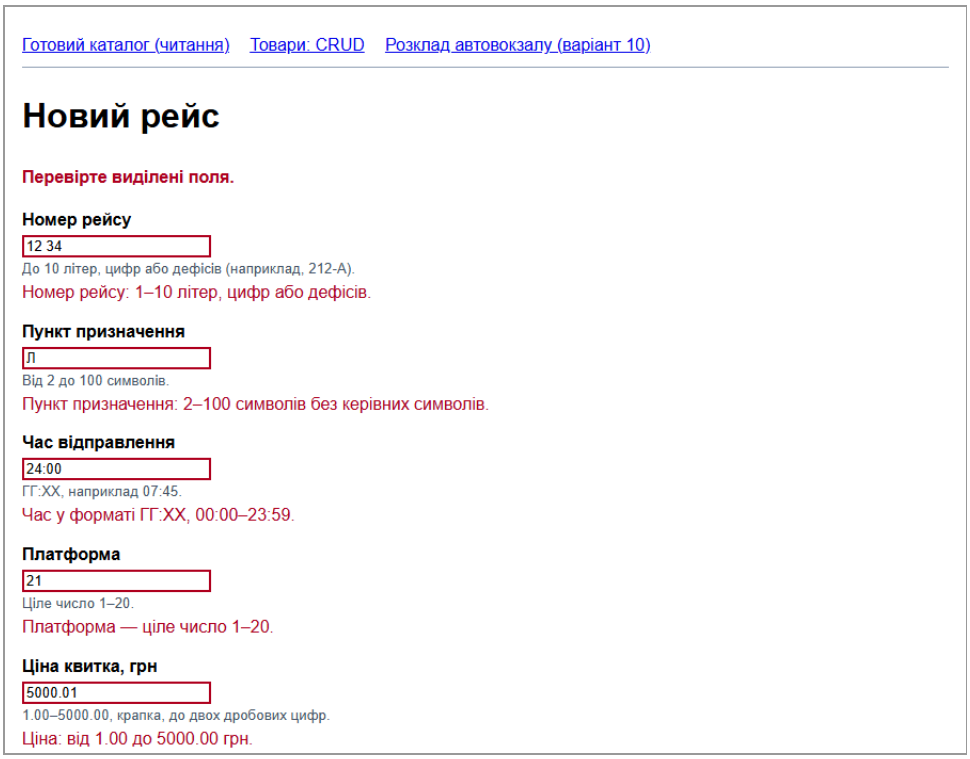


Рис. 15. Перевищення меж: помилка біля кожного поля (номер із пробілом, назва з 1 символу, 24:00, платформа 21, ціна 5000.01).

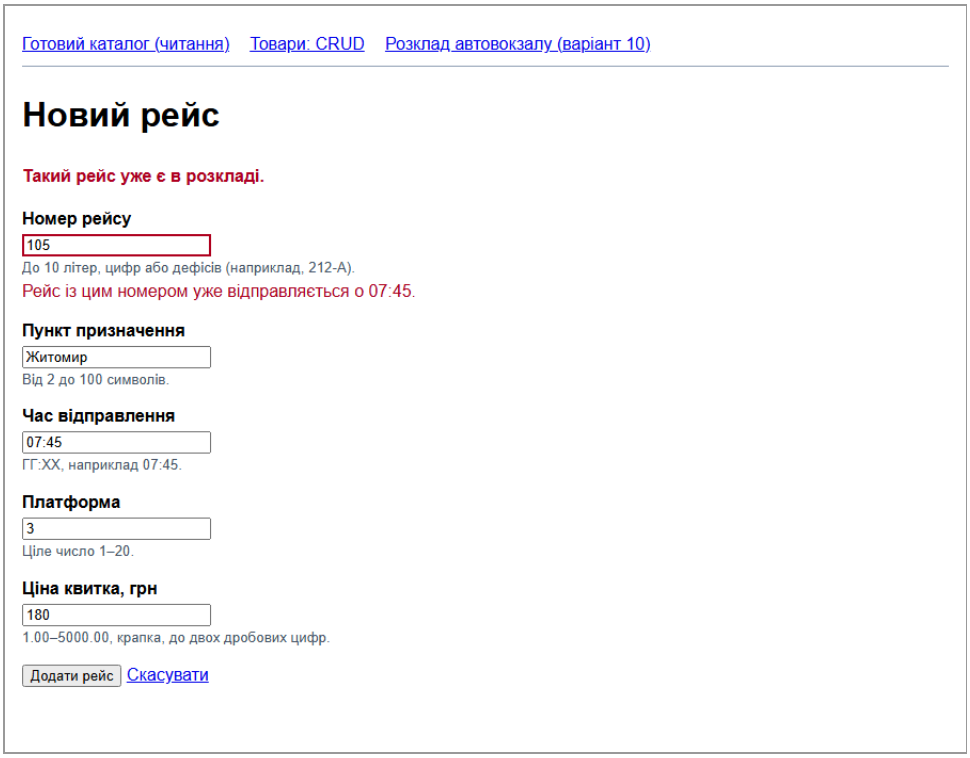


Рис. 16. Рейс 105 о 07:45 уже є — конфлікт унікального індексу показано біля поля «Номер рейсу» (409).

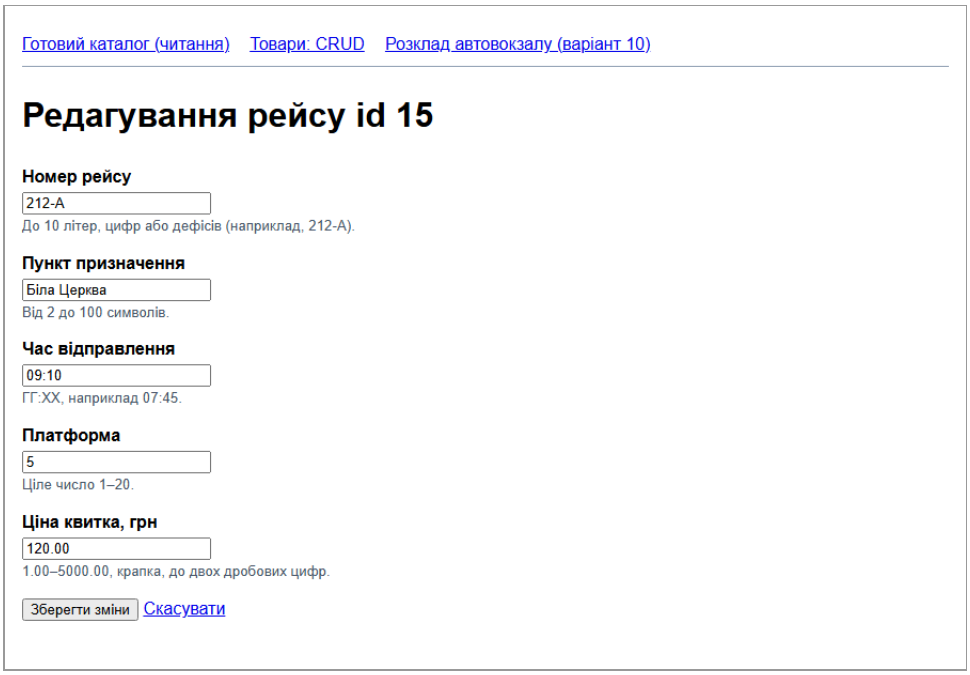


Рис. 17. Форма редагування рейсу за id (дані з таблиці).

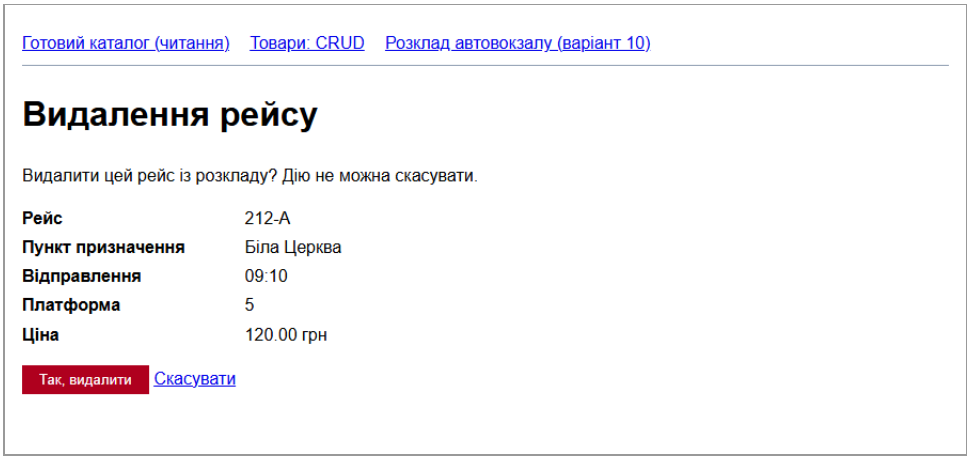


Рис. 18. Підтвердження видалення рейсу.

Перезапуск (8.5). MariaDB зупинено (`mariadb-admin shutdown`) — сайт повернув 500 із нейтральним повідомленням; після запуску MariaDB і PHP-сервера заново `routes.php` показав ті самі 8 рейсів, `index.php` — 3 товари: дані читаються з таблиці.

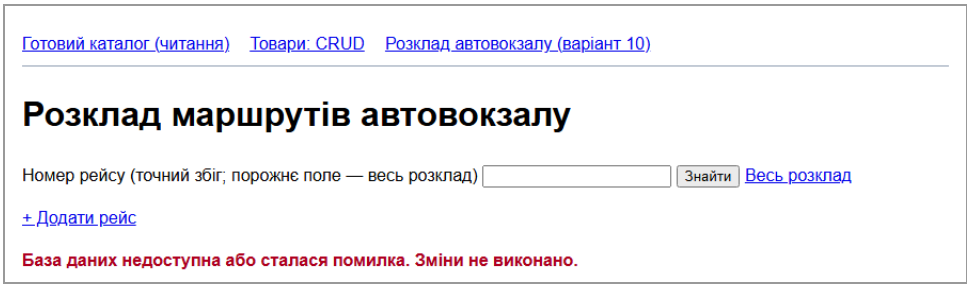


Рис. 19. Неправильний пароль `DB_PASSWORD` (окремий сервер на порту 8001): нейтральне повідомлення, без реквізитів і тексту винятку.

Крок 9. Експорт і відновлення

У phpMyAdmin для бази `lab11` виконано «Експорт» (швидкий метод, формат SQL) → `export/lab11.sql` (структура й дані обох таблиць, ключі, AUTO_INCREMENT). Створено порожню базу `lab11_restore`, файл імпортовано через «Імпорт», після чого порівняно з оригіналом; застосунок також запущено з `DB_NAME=lab11_restore`.

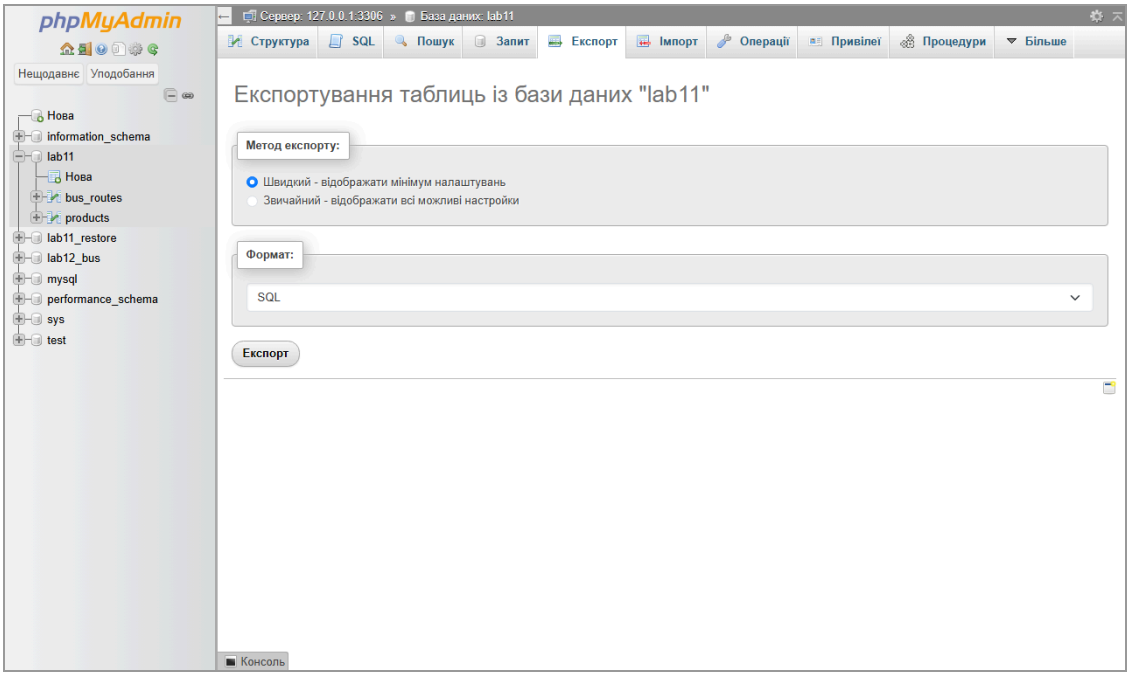


Рис. 20. phpMyAdmin → `lab11` → «Експорт», формат SQL.

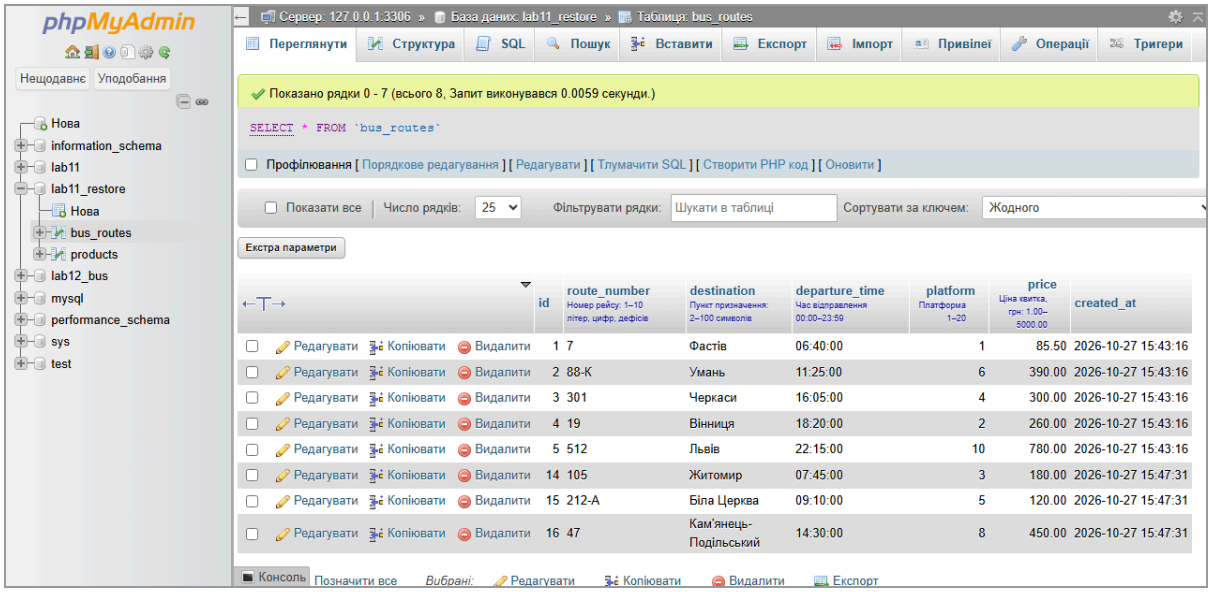


Рис. 21. Відновлена таблиця `lab11_restore.bus_routes`: ті самі 8 рейсів, українські назви й ціни без змін.

```
Крок 9. Експорт phpMyAdmin (export/lab11.sql) → імпорт phpMyAdmin у порожню базу lab11_restore →
порівняння.
Порівняння виконано обліковим записом адміністратора (клієнт mariadb), 27.10.2026.

== Дані (однакові в обох базах)
```

```
                lab11                    lab11_restore
products        3 рядки, SUM(price)=125.75     3 рядки, SUM(price)=125.75
bus_routes      8 рядків, SUM(price)=2565.50   8 рядків, SUM(price)=2565.50
CHECKSUM TABLE  products 1460988797            products 1460988797
                bus_routes 2794061950          bus_routes 2794061950
Перевірка тексту: «Кам'янець-Подільський» 450.00 і «Набір ручок» 60.25 — однакові.

== Ключі (information_schema.statistics) — однакові
products: PRIMARY, uq_products_code
bus_routes: PRIMARY, uq_bus_routes_number_time (route_number, departure_time), idx_bus_routes_destination

== Розбіжність: CHECK-обмеження
lab11: 2 (chk_bus_routes_platform, chk_bus_routes_price); lab11_restore після імпорту: 0.
Причина: phpMyAdmin 5.2.3 записав для bus_routes «CREATE TABLE ... ( ... ) ;» без CHECK і без параметрів
таблиці.
ENGINE і кодування в lab11_restore все одно InnoDB / utf8mb4_unicode_ci (типові значення бази).
Виправлення: виконано sql/routes-checks.sql у lab11_restore → CHECK-обмежень 2.
Після цього SHOW CREATE TABLE обох таблиць (без лічильника AUTO_INCREMENT) у двох базах збігаються
повністю.

== Застосунок із DB_NAME=lab11_restore
routes.php — 8 рейсів, index.php — 3 товари; таблиця рейсів на сторінці ідентична сторінці з lab11.
```

Висновок щодо відновлення. Дані, ключі, кодування й ціни збігаються (однакові `CHECKSUM TABLE`). phpMyAdmin 5.2.3 не переніс два CHECK-обмеження `bus_routes` — тому відновлення перевіряли порівнянням, а не «на довіру». Після `sql/routes-checks.sql` структура збігається повністю.

4. Результати перевірок

Фактичні результати — з `tools/checks-output.txt`, `cli-output.txt`, `restore-check.txt` (додатки А–Б). W3C Nu Validator: 18 станів сторінок застосунку — 0 помилок і попереджень.

Сценарій	Дані	Очікування	Фактичний результат	Висновок
Початковий імпорт	schema.sql у порожню lab11	3 товари, правильні ціни	Зошит 25.50, Папка 40.00, Набір ручок 60.25	Виконано
COUNT і WHERE	`price >= 40.00`	3 / 2	3 / 2	Виконано
Пошук	PD1002; порожньо; NO-SUCH-CODE	1 товар; список; повідомлення	Папка; 3 рядки; «Товарів не знайдено.»	Виконано
Масив у GET	`code[]=PD1002`, `number[]=x`	400 без warning	400 «Код має бути рядком.» / «Номер рейсу має бути рядком.»	Виконано
POST до сторінки читання	POST index.php	405, дані не змінено	405; CHECKSUM TABLE без змін	Виконано
CLI crud-demo	php crud-demo.php	Цикл підтверджено, 3 товари	«…перевірено; транзакцію відкочено», 3 товари	Виконано

Сценарій	Дані	Очікування	Фактичний результат	Висновок
Спецсимволи	`Набір 'Код' & <b>друзі</b>`, `Кам'янець & <Поділля>`	Точне збереження, теги як текст	У БД без змін; у HTML `'`, `&`, `<b>`	Виконано
Допустимі ціни	0; 25; 25.5; 25.50	Нормалізація до 2 знаків	0.00; 25.00; 25.50; 25.50	Виконано
Недопустимі ціни	-1, 25,50, 1e3, 25.555, 100000000	Відмова до зміни БД	422 для всіх; таблиця не змінена	Виконано
Некоректні поля	Без опису; назва з пробілів; `product_name[]`	Помилка без запису	422; без warning/TypeError; таблиця не змінена	Виконано
Повторний код	PD1001; UPDATE на PD1002	Конфлікт, запис не створено	409 «Товар із кодом … уже існує»; 1 товар PD1001	Виконано
UPDATE без змін	Ті самі значення	Коректно при affected_rows = 0	303, «збережено без змін (affected_rows = 0)»	Виконано
UPDATE ціни	0.00 → 45	Змінено	affected_rows = 1; у БД 45.00	Виконано
Невідомий id	id=999999 (товар і рейс), abc, `id[]=1`	Не створюється запис	404 / 400; кількість і CHECKSUM без змін	Виконано
CSRF	Без токена / чужий токен для INSERT, UPDATE, DELETE	Відмова до запиту	403 в усіх випадках; таблиця не змінена	Виконано
Оновлення сторінки	Два GET після 303	INSERT не повторюється	Кількість 8/8	Виконано
Видалення	GET, потім POST; повторний POST	GET не видаляє; повтор — 404	GET 200 із кнопкою, запис є; POST 303; повтор 404	Виконано
Межі рейсу (допустимі)	платформа 1/20, ціна 1.00/5000, 00:00/23:59, 10-символьний номер, 100-символьна назва	Прийнято	303 для всіх	Виконано
Межі рейсу (перевищення)	платформа 0/21, ціна 0.99/5000.01/25,50, 24:00, назва 1/101 символ, номер 11 символів, порожні поля	Відмова	422 з помилкою біля відповідного поля	Виконано
Дублікат рейсу	105 о 07:45	Конфлікт	409, помилка біля «Номер рейсу»	Виконано
Точний пошук рейсу	212-A; 212	1 рейс; нічого	Біла Церква; «Рейс «212» не знайдено.»	Виконано
Імпорт ЛР 8	JSON, потім XML з тими самими рейсами	3 рейси; повтор відкочено	3 імпортовано; XML — 1062, нічого не додано	Виконано
Перезапуск	Зупинка й запуск MariaDB і PHP	Дані з таблиці	8 рейсів і 3 товари після запуску	Виконано

Сценарій	Дані	Очікування	Фактичний результат	Висновок
Збій БД	Неправильний пароль; зупинений сервер	Нейтральна помилка	500 без реквізитів і тексту винятку	Виконано
Відновлення SQL	Експорт phpMyAdmin → lab11_restore	Структура, ключі, записи збігаються	Дані й ключі збігаються; 2 CHECK втрачено експортом, повернено routes-checks.sql	Виконано з доповненням

5. Відповіді на контрольні питання

1. **MariaDB** — сервер СКБД, що зберігає дані й виконує SQL. **phpMyAdmin** — вебінтерфейс адміністрування з власним підключенням до сервера (структура, SQL, імпорт/експорт); він не є посередником для застосунку. **MySQLi** — розширення PHP, через яке код застосунку підключається до MySQL/MariaDB.
2. Ні. Процедурний і об'єктний інтерфейси MySQLi рівноправні й підтримуються. Безпечність визначають підготовлені запити замість конкатенації, перевірка типів і меж, обробка помилок без розкриття реквізитів, мінімальні права користувача й екранування виведення.
3. `PRIMARY KEY` однозначно ідентифікує рядок (унікальний і не NULL); `UNIQUE` забороняє повтор значень (код товару, рейс + час); `NOT NULL` вимагає значення; `AUTO_INCREMENT` генерує id на сервері.
4. `DECIMAL` зберігає десяткові числа точно (гроші), а float — двійкове наближення ($0.1 + 0.2 \neq 0.3$). У PHP ціну передаємо перевіреним десятковим рядком з типом `s`, щоб не пройти через float; MySQLi і повертає DECIMAL рядком.
5. `fetch_assoc` — масив за назвами стовпців; `fetch_row` — за індексами 0…n; `fetch_object` — об'єкт stdClass з властивостями; `fetch_array` з `MYSQLI_BOTH` — обидва види ключів (у демонстрації 8 ключів на 4 стовпці).
6. Результат — курсор: кожен fetch повертає поточний рядок і переходить до наступного; в кінці fetch дає `null`. Щоб прочитати знову, виконують SELECT повторно або для буферизованого результату `mysqli_query` повертають курсор `mysqli_data_seek($result, 0)`.
7. `prepare` надсилає SQL із `?` і отримує підготовлений вираз; `bind_param` прив'язує змінні та їхні типи (`i`, `d`, `s`, `b`); `execute` передає значення й виконує запит; `bind_result` прив'язує змінні до стовпців результату (кількість і порядок як у SELECT); `fetch` заповнює їх рядок за рядком; наприкінці — `stmt_close`.
8. `bind_param` лише прив'язує змінні — запит ще не виконано. Успіх підтверджують `execute` без винятку і `affected_rows` (або повторний SELECT); тільки після цього можна повідомляти про оновлення.
9. `insert_id` — значення AUTO_INCREMENT, створене останнім INSERT на цьому з'єднанні; `affected_rows` — кількість змінених/доданих/видалених рядків. UPDATE змінює 0 рядків, якщо рядка з таким WHERE немає або нові значення збігаються зі старими; тому існування рядка перевіряємо окремо.

10. Транзакція об’єднує кілька змін в одну неподільну дію: `commit` фіксує все, `rollback` скасовує все (імпорт групи рейсів, crud-demo). Лічильник AUTO_INCREMENT не є транзакційним: видане значення не повертається після відкату, тому з’являються пропуски id.
11. Це захист різних меж: параметризація — структуру SQL; валідація — правильність даних (типи, межі, формат); екранування `htmlspecialchars` — HTML-виведення від XSS; CSRF-токен — від підробки запиту з чужого сайту. Жодна з дій не замінює іншу (наприклад, HTML-сутності не захищають SQL).
12. Відновити файл в іншій порожній базі й порівняти з оригіналом: кількість рядків, контрольні суми (`CHECKSUM TABLE`), ключі й обмеження (`SHOW CREATE TABLE`, `information_schema`), український текст і ціни, а також роботу застосунку з новою базою. У роботі саме так виявлено втрату CHECK-обмежень.

6. Висновок

У роботі створено таблицю й виконано SELECT, INSERT, UPDATE, DELETE через phpMyAdmin і процедурний MySQLi; реалізовано вебформи CRUD для товарів і для власної таблиці розкладу автовокзалу (варіант 10) з перенесенням каталогу ЛР 8 у транзакції. Усі запити з введенням — підготовлені, дані змінює лише POST із CSRF-токеном, конфлікти унікальності й збої БД обробляються зрозуміло й без розкриття реквізитів. Експорт phpMyAdmin відновлено в іншій базі й перевірено порівнянням; виявлену втрату CHECK-обмежень усунено окремим SQL-файлом.

7. Посилання на репозиторій

Репозиторій: https://github.com/mykhailokrainiak/web-progr
Папка з лабораторною роботою: https://github.com/mykhailokrainiak/web-progr/tree/main/lab11

Додаток А. Вивід tools/checks.sh

```
== Початковий стан
товарів: 3, рейсів: 8
COUNT(*) і WHERE price >= 40.00: 3 / 2

== Готова сторінка index.php
список: 200 / рядків 3
code=PD1002: 200 /  PD1002 Папка
code=NO-SUCH-CODE: 200 Товарів не знайдено.
code[]=PD1002: 400 Код має бути рядком.
code=pd1002 (малі літери): 400 Неприпустимий формат коду.
POST до index.php: 405 Ця сторінка підтримує лише GET.; таблиця не змінено

== Товари: додавання й спецсимволи
апостроф, & і <b>, ціна 0: 303 → products.php
   у БД: Набір 'Код' & <b>друзі</b> | Опис <i>курсивом</i> | 0.00
   у HTML: Набір &#039;Код&#039; &amp; &lt;b&gt;друзі&lt;/b&gt;
ціна 25: 303 → БД: 25.00
```

```
ціна 25.5: 303 → БД: 25.50
ціна 25.50: 303 → БД: 25.50
ціна -1: 422 Ціна: 0–99999999.99, крапка, до двох дробових цифр.
ціна 25,50: 422 Ціна: 0–99999999.99, крапка, до двох дробових цифр.
ціна 1e3: 422 Ціна: 0–99999999.99, крапка, до двох дробових цифр.
ціна 25.555: 422 Ціна: 0–99999999.99, крапка, до двох дробових цифр.
ціна 100000000: 422 Ціна: 0–99999999.99, крапка, до двох дробових цифр.
   таблиця не змінено
без product_desc: 422 Усі поля товару мають бути рядками UTF-8.
назва з пробілів: 422 Назва: 1–80 символів, опис: 1–500 символів.
product_name[] масивом: 422 Усі поля товару мають бути рядками UTF-8.
   таблиця не змінено
повторний код PD1001: 409 Товар із кодом «PD1001» уже існує. Вкажіть інший код.; значення у формі
збережено: 1
   товарів із кодом PD1001: 1; таблиця не змінено

== Товари: редагування
GET форма id=11: 200
UPDATE тими самими даними: 303 → products.php
   Товар id 11 збережено без змін (affected_rows = 0).
UPDATE ціни на 45: 303 → products.php
   Товар id 11 оновлено (affected_rows = 1).; БД: 45.00
UPDATE на чужий код PD1002: 409 Товар із кодом «PD1002» уже існує. Вкажіть інший код.
POST невідомий id=999999: 404 Товар не знайдено; новий запис не створено.
GET невідомий id: 404 Товар не знайдено.; id=abc: 400 Некоректний id товару.; id[]=1: 400 Некоректний id
товару.
   товарів 7 → 7; таблиця не змінено

== CSRF (товари)
INSERT без токена: 403 Форма застаріла (CSRF-токен не збігається). Оновіть сторінку.
INSERT із чужим токеном: 403 Форма застаріла (CSRF-токен не збігається). Оновіть сторінку.
UPDATE без токена: 403 Форма застаріла (CSRF-токен не збігається). Оновіть сторінку.
DELETE без токена: 403 Форма застаріла (CSRF-токен не збігається). Оновіть сторінку.
   таблиця не змінено

== Оновлення сторінки після збереження
POST: 303 → products.php
товарів до/після двох GET: 8/8

== Видалення з підтвердженням
TEST-APOS (id 11): GET 200, кнопка підтвердження: 1, після GET запис є: 1; POST 303 → products.php
TEST-P25 (id 12): GET 200, кнопка підтвердження: 1, після GET запис є: 1; POST 303 → products.php
TEST-P25-5 (id 13): GET 200, кнопка підтвердження: 1, після GET запис є: 1; POST 303 → products.php
TEST-P25-50 (id 14): GET 200, кнопка підтвердження: 1, після GET запис є: 1; POST 303 → products.php
TEST-PRG (id 16): GET 200, кнопка підтвердження: 1, після GET запис є: 1; POST 303 → products.php
повторне видалення id 11: 404 Товар не знайдено; нічого не видалено.
товарів після очищення: 3 (очікується 3)

== Варіант 10: розклад (bus_routes)
список: 200 / рядків 8
точний пошук 212-A: 200 / рядків 1: Біла Церква
пошук 212 (не точний збіг): 200 Рейс «212» не знайдено.
number[]=x: 400 Номер рейсу має бути рядком.; number=a%20b: 400 Номер рейсу: 1–10 літер, цифр або дефісів.
додавання (спецсимволи): 303 → routes.php
   у БД: T-1 | Кам'янець & <Поділля> | 05:00:00 | 1 | 1.00
   у HTML: Кам&amp;#039;янець &amp;amp; &amp;lt;Поділля&amp;gt;
Межі (UPDATE того самого рейсу id 12):
  ✓ платформа 20, ціна 5000, 23:59: 303
  ✓ ціна 1.00, час 00:00: 303
  ✓ номер 10 символів, назва 100 символів: 303
  ✗ платформа 0: 422 platform: Платформа — ціле число 1–20.
  ✗ платформа 21: 422 platform: Платформа — ціле число 1–20.
  ✗ ціна 0.99: 422 price: Ціна: від 1.00 до 5000.00 грн.
  ✗ ціна 5000.01: 422 price: Ціна: від 1.00 до 5000.00 грн.
  ✗ ціна 25,50: 422 price: Ціна: число з крапкою, до двох дробових цифр (наприклад, 180 або 85.50).
  ✗ час 24:00: 422 departure_time: Час у форматі ГГ:ХХ, 00:00–23:59.
  ✗ назва 1 символ: 422 destination: Пункт призначення: 2–100 символів без керівних символів.
  ✗ назва 101 символ: 422 destination: Пункт призначення: 2–100 символів без керівних символів.
  ✗ номер 11 символів: 422 route_number: Номер рейсу: 1–10 літер, цифр або дефісів.
  ✗ порожні поля: 422 route_number: Заповніть поле.;destination: Заповніть поле.;departure_time: Заповніть
поле.;platform: Заповніть поле.;price: Заповніть поле.
```

```
дублікат рейсу 105 о 07:45: 409 Такий рейс уже є в розкладі. | route_number: Рейс із цим номером уже
відправляється о 07:45.
CSRF (рейс): 403 Форма застаріла (CSRF-токен не збігається). Оновіть сторінку.
редагування невідомого id: 404 Рейс не знайдено; новий запис не створено.
видалення тестового рейсу: GET 200 / POST 303 → routes.php
рейсів після очищення: 8 (очікується 8)

== Неправильні реквізити БД (сервер :8001)
index.php: 500 Каталог недоступний. Перевірте налаштування та журнал PHP.
products.php: 500 База даних недоступна або сталася помилка. Зміни не виконано.
routes.php: 500 База даних недоступна або сталася помилка. Зміни не виконано.
POST додавання: 500 База даних недоступна або сталася помилка. Зміни не виконано.
```

Додаток Б. Вивід консольних сценаріїв (tools/cli-output.txt)

```
$ php crud-demo.php
INSERT → SELECT → UPDATE → SELECT → DELETE перевірено; транзакцію відкочено.

$ php fetch-demo.php
mysqli_fetch_assoc — доступ за назвою стовпця:
  1: Зошит — 25.50
  2: Папка — 40.00
  3: Набір ручок — 60.25
  ще один fetch після циклу: NULL (курсор у кінці)

mysqli_fetch_row — доступ за індексом (назва — третій стовпець, [2]):
  1: Зошит — 25.50
  2: Папка — 40.00
  3: Набір ручок — 60.25

mysqli_fetch_object — властивості об’єкта stdClass:
  1: Зошит — 25.50
  2: Папка — 40.00
  3: Набір ручок — 60.25

mysqli_fetch_array(MYSQLI_BOTH) — і назва, і індекс:
  ['product_name'] = Зошит, [2] = Зошит, ключів у рядку: 8 (4 стовпці × 2)
  типи значень: id — string, price — string (DECIMAL повертається рядком)

Порожній результат: mysqli_num_rows = 0, fetch = NULL — це не помилка SQL.

$ php routes-import.php json
Імпортовано з routes-lab08.json: 3 рейс(и); транзакцію підтверджено.
(код завершення 0)

$ php routes-import.php xml   # ті самі рейси повторно
Рейс із таким номером і часом уже є в таблиці; транзакцію відкочено, нічого не імпортовано.
(код завершення 1)
```